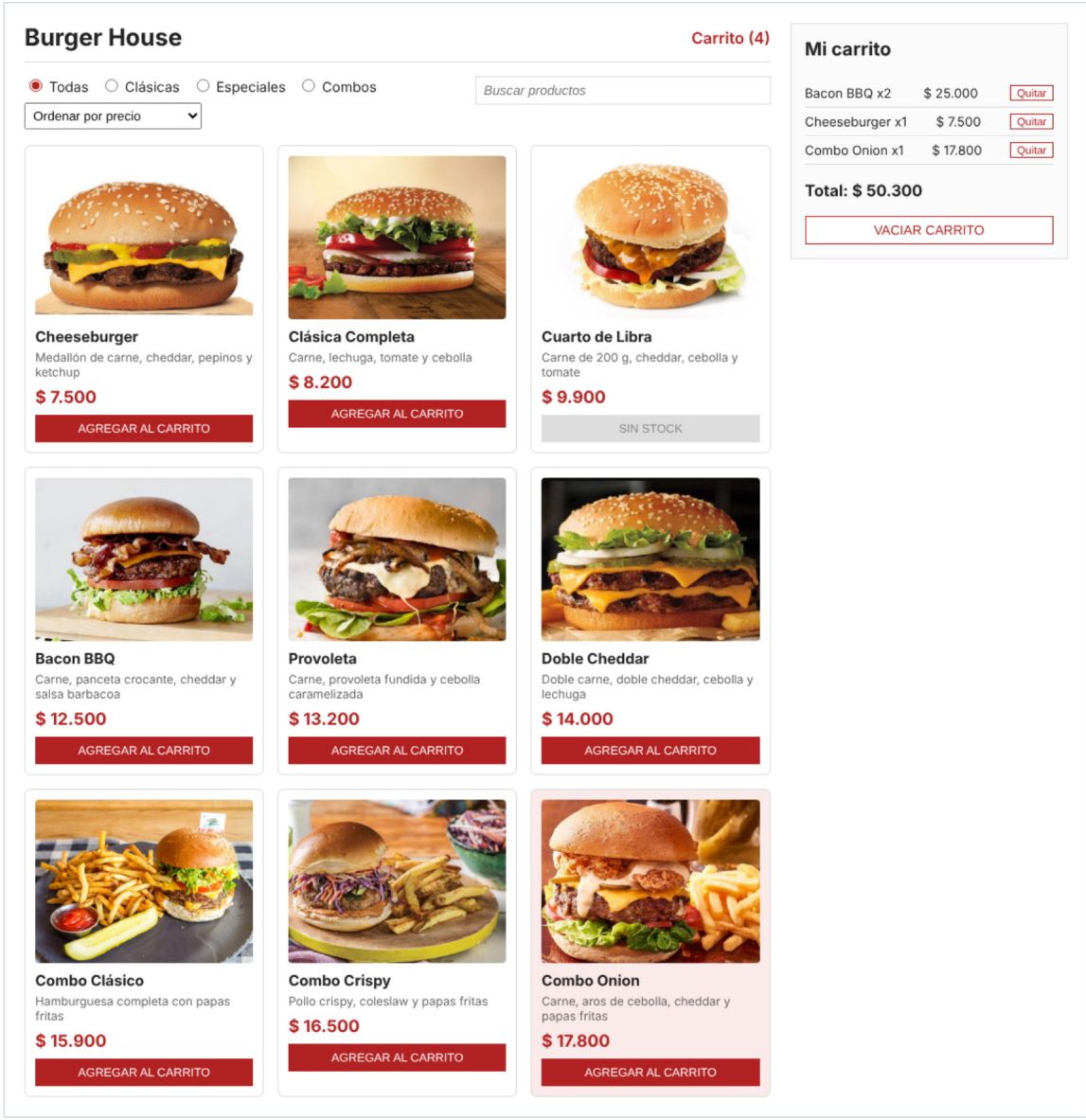
Armando una tienda online

¡Vamos a emular una prueba técnica!

Tendrán que clonar la siguiente imagen de muestra, correspondiente al archivo listaProductos.png . El resultado final debe ser similar al de la imagen.

Los datos de las hamburguesas están en index.js y las fotos en la carpeta img/. Cada producto tiene id, nombre, descripcion, categoria, precio, stock e imagen.



Parte 1 / HTML

- **1 punto** / Armar toda la estructura HTML con las etiquetas semánticas fundamentales. Incluyendo un <header> donde pondrán *Parcial 1 y la División que corresponda* y el código correspondiente a la imagen en la etiqueta <main>
- En el <footer> escriban un párrafo donde pondrán su nombre y apellido
- **1 punto** / Identifiquen y clonen las etiquetas usadas en los elementos de la imagen
  - El título *Burger House* y, a su derecha, el contador *Carrito (0)*
  - La barra horizontal
  - Los inputs de *Todas, Clásicas, Especiales, Combos*
  - La barra de búsqueda
  - El select para ordenar por precio (*Menor precio / Mayor precio*)
  - Un section vacío con id products-container (posteriormente lo llenarán desde JavaScript)
  - Un aside con el título *Mi carrito*, una lista vacía con id cart-list, el total con id cart-total y el botón *VACIAR CARRITO*

Parte 2 / CSS

- **1 punto** / Maquetar con flex (o grid si prefieren) la página para que todo se vea igual que en la imagen: el catálogo a la izquierda, el carrito a la derecha y los productos en una grilla de 3 columnas. Además, el texto del header y del footer debe estar centrado sin usar flex
- **0.25 puntos** / Crear (y usar) variables CSS para almacenar los siguientes colores
  - Color primario: firebrick o #B22222
  - Color gris oscuro: #dddddd
  - Color gris claro: #fbfbfb
  - Color secundario: #fbeaea
- **0.5 puntos** / Usen el color primario para el precio de cada producto y el fondo de los botones *Agregar al carrito*. Usen el color gris oscuro para la barra horizontal, el borde de la barra de búsqueda y el borde de las tarjetas, y el color gris claro para el fondo del carrito.
- **0.25 puntos** / Usen el color secundario para que al pasar el mouse por una tarjeta de producto tenga ese color de fondo. Los botones deshabilitados (*Sin stock*) deben verse en gris.

Parte 3 / JavaScript

- **1 punto** / Trabajar y guardar localmente los datos con JSON

  i. Conviertan a un array de objetos el json de index.js

  ii. Guárdenlos en localStorage con la clave productos.

  iii. Obtengan estos productos directamente del localStorage y muéstrenlos en un console.table

- **1.5 puntos** / Manipulen el DOM para que al hacer click en el título *Burger House*, se rendericen todos los productos como tarjetas dentro de products-container . Cada tarjeta debe tener:

  - La imagen del producto.
  - El nombre y la descripción.
  - El precio.
  - Un botón *Agregar al carrito*. Si el producto tiene stock: 0, el botón debe decir *Sin stock* y estar deshabilitado.

Parte 3 / JavaScript

- **1.5 puntos /** Agreguen la funcionalidad de los filtros. Los tres deben poder combinarse entre sí:
  - Usen el evento keyup sobre el buscador: mostrar solo los productos cuyo nombre o descripción contengan el texto ingresado, sin distinguir mayúsculas y minúsculas
  - Ordenar por precio, con sort sobre una copia del array
  - Si ningún producto coincide, muestren el mensaje "No se encontraron productos"

- **2 puntos /** Implementen el carrito de compras:
  - **Agregar:** suma el producto con cantidad 1; si ya estaba, aumenta su cantidad sin superar el stock (si se supera, un alert con "No hay más stock disponible")
  - **Sacar:** cada producto se muestra en cart-list con nombre, cantidad, subtotal y un botón *Quitar*, que resta 1; si llega a 0, se elimina del carrito
  - **Vaciar:** el botón *VACIAR CARRITO* elimina todos los productos
  - Cada vez que el carrito cambie, actualicen cart-total, el contador *Carrito (n)* y guarden el carrito en localStorage con la clave carrito . Si está vacío, muestren "Tu carrito está vacío"